Encapsulamiento y Modificadores de Acceso en Java

Taller Práctico: Simulador de Batalla RPG

1. Presentación y Contexto del Taller

1.1. Contextualización de la Misión RPG

Estimados estudiantes, bienvenidos a la misión práctica de la Unidad 2.2. En este taller asumirán el rol de ingenieros de software encargados de diseñar e implementar el motor central de combate para un videojuego de consola titulado "**Simulador de Batalla / Arena de Héroes**".

El objetivo fundamental de esta práctica es garantizar el control de estado y la integridad de los datos de nuestros entes de juego mediante la aplicación rigurosa del principio de **Encapsulamiento** y el uso estratégico de los **Modificadores de Acceso** en Java.

Al finalizar esta guía, el estudiante habrá logrado:

- Aplicar visibilidad restringida (`private`) a los atributos centrales de una clase para evitar la manipulación no autorizada del estado interno.
- Exponer una interfaz pública de métodos (`public`) que actúe como un contrato seguro de interacción con el exterior.
- Diseñar e implementar métodos mutadores (`setters`) con lógica de control rigurosa para la validación de reglas de negocio.
- Comprender las barreras de acceso entre paquetes mediante la segregación de clases del dominio.
- Integrar estructuras de control previas (`do-while` , `switch` , `if-else`) con formateo profesional de cadenas de texto (`System.out.printf` , `StringBuilder`).

1.2. Requisitos de Entorno y Arquitectura General

Para el desarrollo satisfactorio del taller, se requiere el siguiente entorno de trabajo:

- **Entorno Integrado de Desarrollo (IDE):** IntelliJ IDEA (versión Community o Ultimate).
- **Kit de Desarrollo de Java (JDK):** Versión 17 o superior correctamente configurada en la variable de entorno `JAVA_HOME` .

La arquitectura del proyecto debe estructurarse combinando los conceptos teóricos abordados en las Unidades 1.1 a 1.4 y 2.1 a 2.2, distribuidos en la siguiente estructura de paquetes:

```
src/
└── co/edu/puj/rpg/
    ├── modelo/
    │   ├── Personaje.java
```

```
    |   └── enemigo/
    |       └── Enemigo.java
    └── principal/
        └── Main.java
```

2. Sección 1: Diseño de la Clase Encapsulada (`Personaje` y `Enemigo`)

2.1. Definición Rigurosa de Atributos Privados

La clase `Personaje` debe encapsular de manera absoluta los datos del héroe. Ningún atributo de la clase debe ser accesible directamente desde clases externas o el método principal.

A continuación, se especifican los atributos que deben ser declarados estrictamente con el modificador de acceso `private`:

Atributo	Tipo de Dato	Modificador de Acceso	Propósito
`nombre`	`String`	`private`	Almacena el nombre del personaje.
`puntosVida`	`int`	`private`	Representa la cantidad de salud actual del personaje en combate.
`vidaMaxima`	`int`	`private`	Determina el límite máximo absoluto de salud que el personaje puede alcanzar.
`puntosAtaque`	`int`	`private`	Valor base de daño infligido por el personaje en cada turno.
`nivel`	`int`	`private`	Nivel de progresión del personaje en la arena.
`oro`	`double`	`private`	Saldo monetario acumulado para transacciones en la tienda.
`estaVivo`	`boolean`	`private`	Estado lógico que indica si el personaje permanece consciente o ha sido derrotado.

Requisito de Arquitectura entre Paquetes: El estudiante deberá replicar una estructura de atributos privados dentro de una clase secundaria denominada `Enemigo`, la cual debe ser ubicada estrictamente dentro del subpaquete `co.edu.puj.rpg.modelo.enemigo`. Esta distribución permitirá verificar el aislamiento de acceso que imponen los visores por defecto (*package-private*) y `private` cuando se intenta realizar una interacción directa desde la clase `Main` (ubicada en `co.edu.puj.rpg.principal`).

2.2. Especificación de Constructores y Firma de Métodos

El estudiante debe implementar dos constructores en la clase `Personaje`:

1. Un constructor sobrenatural/completo que inicialice el `nombre`, `vidaMaxima`, `puntosAtaque` y `oro`.

2. El constructor debe hacer uso explícito de la palabra clave `this` para desambiguar formalmente los parámetros de entrada de los atributos de la instancia.

Las firmas de los métodos de la clase `Personaje` deben mantener la siguiente plantilla de interfaz, **sin incorporar código de implementación**:

```
public class Personaje {

    // Declaración de atributos privados...

    // Constructores
    public Personaje(String nombre, int vidaMaxima, int puntosAtaque, double oro) {
        // Implementación requerida por parte del estudiante usando 'this'
    }

    // Firmas de la API pública de comportamiento
    public void atacar(Enemigo enemigo) {
        // Estructura requerida para procesar el ataque a la entidad enemiga
    }

    public void usarPocion(int cantidadRestaurada) {
        // Estructura requerida para la recuperación de salud
    }

    public void recibirDano(int cantidadDano) {
        // Estructura requerida para reducir puntos de vida mediante el setter
correspondiente
    }
}
```

3. Sección 2: Implementación de Setters Validados y Getters

3.1. Especificación de Métodos Mutadores (`Setters`) con Lógica de Control

El acceso de escritura a los atributos privados debe realizarse exclusivamente a través de métodos mutadores (`setters`). El estudiante debe codificar la siguiente lógica de validación explícita para evitar inconsistencias en el estado del objeto:

1. `**setPuntosVida(int vida)**` :
   - Si el valor ingresado es menor o igual a `0` , la variable `puntosVida` debe asignarse estrictamente a `0` , y el atributo `estaVivo` debe cambiar automáticamente a `false` .
   - Si el valor ingresado es superior a `vidaMaxima` , la variable `puntosVida` no puede exceder el límite establecido por `vidaMaxima` .

- Para cualquier valor intermedio válido, se asigna el valor recibido y se garantiza que `estaVivo` permanezca en `true`.

2. `**setOro(double oro)**`:
   - El valor de `oro` no puede registrar magnitudes negativas bajo ninguna circunstancia. Si una transacción financiera genera un monto menor a `0.0`, la operación debe ser rechazada e indicar un saldo insuficiente sin alterar el atributo.
3. `**setNivel(int nivel)**`:
   - Se debe restringir la asignación del nivel impidiendo cualquier valor inferior a `1`. En caso de recibir un valor inválido, el atributo debe mantener su valor actual.

3.2. Métodos Lectores (`Getters`) y API Pública de la Clase

Para permitir la inspección segura del estado del personaje desde el bucle principal de juego, el estudiante deberá implementar la totalidad de los métodos lectores (`getters`) correspondientes:

- `getNombre()` : Retorna el nombre (`String`).
- `getPuntosVida()` : Retorna los puntos de vida actuales (`int`).
- `getVidaMaxima()` : Retorna el valor tope de salud (`int`).
- `getPuntosAtaque()` : Retorna el nivel de daño base (`int`).
- `getNivel()` : Retorna el nivel del héroe (`int`).
- `getOro()` : Retorna el saldo en oro (`double`).
- `isEstaVivo()` : Retorna el estado lógico del personaje (`boolean`).

Todos estos métodos deben contar estrictamente con el modificador de acceso `public`, constituyendo la API de lectura oficial que consumirá la clase `Main`.

4. Sección 3: Construcción de la Lógica de Batalla en Consola

4.1. Bucle Principal y Menú Interactivo

En la clase `Main` (método `main`), el estudiante implementará la interfaz gráfica por consola. La interacción del combate debe estar gobernada por las siguientes directrices:

- Utilizar un bucle `do-while` cuya condición de permanencia evalúe que tanto el `Personaje` como el `Enemigo` mantengan su atributo `estaVivo` en `true`.
- Desplegar un menú interactivo dentro del bucle utilizando la estructura de selección `switch` con las siguientes opciones:
  1. **Atacar**: Invoca el método `atacar()`.
  2. **Usar Poción**: Invoca el método `usarPocion()`.
  3. **Ver Estado**: Muestra los atributos actualizados mediante la API de *getters*.
  4. **Huir**: Rompe la ejecución del combate terminando el bucle.
- Implementar validaciones de entrada errónea mediante bloques `if-else` para gestionar opciones inválidas ingresadas por teclado sin interrumpir la ejecución del programa.

4.2. Formateo de Salida y Registro de Batalla (Logs)

El taller requiere la correcta utilización de mecanismos de manejo de cadenas:

- **Salida Formateada:** La pantalla de estado del combate y los indicadores de salud, oro y nivel deben presentarse empleando `System.out.printf`. Se exige el uso adecuado de especificadores de formato (`%s` para cadenas, `%d` para enteros y `%.2f` para valores flotantes/dobles).
- **Historial de Batalla:** Se debe instanciar un objeto de la clase `StringBuilder` en el flujo de combate para concatenar secuencialmente cada evento (daño infligido, curaciones, cambios de estado). Al finalizar cada ronda, se debe imprimir en consola el historial consolidado acumulado en el `StringBuilder`.

5. Sección 4: Retos de Expansión

5.1. Reto 1: Sistema de Tienda y Gestión Financiera

El estudiante deberá diseñar e incorporar un submenú para una tienda de artículos dentro de la consola:

- Se debe ofrecer la compra de "Pociones Grandes de Salud".
- El proceso de compra debe llamar obligatoriamente al método mutador `setOro()`, pasando el nuevo saldo calculado tras restar el costo de la poción.
- El sistema de la tienda debe rechazar la transacción si el método mutador detecta que el saldo caería a un valor negativo, impidiendo la entrega del ítem al inventario del personaje.

5.2. Reto 2: Calibración de Experiencia y Golpes Críticos

Para incrementar la complejidad del dominio de juego, se deben implementar dos mecanismos adicionales:

1. **Sistema de Experiencia y Subida de Nivel:** Al derrotar a un enemigo, el personaje ganará puntos de experiencia. Al superar un umbral predefinido, se debe incrementar el nivel del héroe invocando el método protegido `setNivel()`.
2. **Sistema de Golpes Críticos:** Modificar la ejecución del método `atacar()` para integrar la generación de números aleatorios (`Math.random()`). Si se genera un evento crítico (probabilidad del 15%), el ataque duplicará su magnitud antes de aplicarse al enemigo.

6. Sección 5: Depuración de Encapsulamiento en IntelliJ IDEA

6.1. Casos de Estudio: Violaciones de Encapsulamiento

A continuación, se presentan tres fragmentos de código que contienen errores de diseño conceptual o infracciones de las reglas de visibilidad en Java. El estudiante debe transcribir estos escenarios a su proyecto en IntelliJ IDEA, observar los mensajes del compilador e identificar la causa raíz de la falla:

Caso A: Acceso Directo no Autorizado (Error de Compilador)

```
public class Main {
    public static void main(String[] args) {
        Personaje heroe = new Personaje("Aragorn", 100, 15, 50.0);

        // Intento ilícito de modificación directa de un atributo privado
        heroe.puntosVida = 200;
        System.out.println("Vida: " + heroe.puntosVida);
    }
}
```

Caso B: Bypassing de Validaciones mediante Constructor Defectuoso

```
public class Personaje {
    private int puntosVida;
    private boolean estaVivo;

    public Personaje(int puntosVida) {
        // Error de encapsulamiento: asignación directa sin pasar por setter o validación
        this.puntosVida = -50;
        this.estaVivo = true; // Inconsistencia: el personaje tiene vida negativa pero está vivo
    }
}
```

Caso C: Fuga de Estado y Asignación fuera de Límites

```
public class Main {
    public static void ejecutarAccion(Personaje heroe) {
        // Mutación que viola las reglas de negocio si el setter no está validado
        heroe.setPuntosVida(999999); // Excede vidaMaxima
        heroe.setPuntosVida(-100);   // Vida negativa sin actualizar estaVivo a false
    }
}
```

6.2. Guía de Verificación en Entorno Integrado

Para validar que el diseño cumple con los estándares de encapsulamiento e integridad solicitados, el estudiante debe realizar la siguiente comprobación analítica en IntelliJ IDEA:

1. Verificar que al escribir `heroe.` desde la clase `Main`, el autocompletado del IDE únicamente despliegue los métodos `public` (getters, setters y métodos de comportamiento) y no muestre

ninguno de los campos de datos privados.

2. Comprobar que cualquier intento de acceso directo a un miembro privado reporte en la ventana de inspección de IntelliJ IDEA el error: `'atributo' has private access in 'co.edu.puj.rpg.modelo.Personaje'`.
3. Validar mediante pruebas unitarias manuales que al asignar valores límite (`-10`, `0`, `100000`) a través de los *setters*, el objeto preserve la coherencia interna de sus atributos en todo momento.